

QUIZ 3

BSCS FALL 2024

CLO MAPPED : CLO 2 & CLO 4

Question :

You are building a recipe app using Expo and want to fetch recipe data from an API endpoint and store it in a global context for use across different components.

Consider the following tasks you need to perform:

1. Fetch data from `https://api.example.com/recipes` when the component mounts.
2. Store the fetched recipe data in a global state using `useContext` to access it in various components of your app

HINT : LOCAL STATE OF DATA WILL BE MADE IN APP.JS AND BOTH STATE VARIABLE AND STATE FUNCTION WILL BE PASSED TO CONTEXT PROVIDER. IT WILL THEN BE USED WHERE API IS CALLED.USE OBJECT Destructuring FOR THIS PURPOSE.

SOLUTION

```
// DataContext.js
```

```
import React, { createContext, useState } from 'react';
```

```
export const DataContext = createContext();
```

```
export const DataProvider = ({ children }) => {
```

```
  const [data, setData] = useState(null);
```

```
  return (
```

```
    <DataContext.Provider value={{ data, setData }}>
```

```
      {children}
```

```
    </DataContext.Provider>
```

```
  );
```

```
};
```

```
.....
```

```
// YourComponent.js
```

```
import React, { useEffect, useContext } from 'react';
```

```
import { DataContext } from '../DataContext';
```

```
import { View, Text, ActivityIndicator } from 'react-native';
```

```

const YourComponent = () => {
  const { data, setData } = useContext(DataContext);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch('https://api.example.com/recipes ');
        const result = await response.json();
        setData(result);
      } catch (error) {
        console.error('Error fetching data:', error);
      }
    };

    fetchData();
  }, []); // Empty dependency array to run only once on mount

  return (
    <View>
      {data ? (
        <Text>{JSON.stringify(data)}</Text>
      ) : (
        <ActivityIndicator size="large" color="#0000ff" />
      )}
    </View>
  );
};

export default YourComponent;

```